



南開大學
Nankai University

计算机学院
并行程序设计实验报告

Pthread & OpenMP 多线程编程

姓名：王一诺

学号：2312331

专业：计算机科学与技术

2025 年 5 月 28 日

目录

1 实验目标	2
2 实验平台配置	2
3 Pthread 多线程编程	2
3.1 朴素 NTT 的 Pthread 多线程优化	2
3.1.1 设计思想	2
3.1.2 代码实现	2
3.1.3 性能分析	5
3.2 CRT 合并的 Pthread 多线程优化	5
3.2.1 多模数 CRT 合并 NTT	5
3.2.2 NTT_CRT_4mod 的 Pthread 多线程优化	7
3.2.3 性能分析	8
4 OpenMP 多线程编程	9
4.1 朴素 NTT 的 OpenMP 多线程优化	9
4.1.1 设计思想	9
4.1.2 代码实现	9
4.1.3 性能分析	10
4.2 CRT 合并的 OpenMP 多线程优化	10
4.2.1 设计思想	10
4.2.2 代码实现	11
4.2.3 性能分析	12
5 实验总结和思考	12
6 链接	12

1 实验目标

本实验旨在探索数论变换（NTT）在多项式乘法中的多线程优化方法，重点关注两种并行化场景：

朴素多线程优化：通过将 NTT 蝶形运算的循环层划分到多个线程，利用线程级并行性加速计算。蝶形运算的独立性使得不同线程可处理不同数据块，通过同步机制保障运算正确性。

多模数 CRT 合并优化：针对超大模数（如超过 64 位）的计算场景，将模数分解为多个小模数，在每个模数上并行执行 NTT，最后通过中国剩余定理（CRT）合并结果。该方法在同态加密等需要大模数的应用中具有显著优势。

实验覆盖 Pthread 和 OpenMP 两种多线程编程模型，通过对比分析不同优化策略的性能提升效果。

2 实验平台配置

平台 \ 处理器	操作系统	编译器
ARM/华为鲲鹏处理器	OpenEuler	GCC

表 1: 实验环境

3 Pthread 多线程编程

3.1 朴素 NTT 的 Pthread 多线程优化

3.1.1 设计思想

在迭代 NTT 的蝶形运算中，每层循环的段（segments）之间相互独立。通过将段分配给不同线程，实现并行计算。具体地，外层循环（h）按蝶形运算的层级展开，内层循环（seg）划分到多个线程，每个线程处理连续的若干段。通过 pthread_barrier 保证每层计算完成后同步所有线程，确保数据一致性。

3.1.2 代码实现

在 NTT_Iteration_Pthread 函数中，首先对输入进行位逆序重排，随后创建多个线程，每个线程执行 butterfly_worker 函数并处理若干段的蝶形操作。线程通过 pthread_barrier_wait 同步进入下一层。乘法函数 NTT_Iteration_multiply_Pthread 在完成正变换、点值乘与逆变换后，归一化输出。

朴素 NTT 的 Pthread 多线程优化

```
1  const int MAX_THREADS = 8;
2  struct ThreadArgs {
3      LL *a;
4      int len;
5      int op;
6      LL g;
7      LL p;
8      int thread_id;
9      int num_threads;
```

```

10     pthread_barrier_t *barrier;
11 };
12
13 void* butterfly_worker(void* args) {
14     ThreadArgs* t = (ThreadArgs*)args;
15     LL *a = t->a;
16     int len = t->len;
17     int op = t->op;
18     LL g = t->g;
19     LL p = t->p;
20     int id = t->thread_id;
21     int T = t->num_threads;
22     auto barrier = t->barrier;
23
24     // 每层 h
25     for (int h = 2; h <= len; h <<= 1) {
26         // 当前层段数
27         int segments = len / h;
28         // 均分到各线程
29         int chunk = (segments + T - 1) / T;
30         int start_seg = id * chunk;
31         int end_seg = std::min(start_seg + chunk, segments);
32         // 计算本层根
33         LL root = (op == 1 ? g : qpowl(g, p - 2, p));
34         LL wn = qpowl(root, (p - 1) / h, p);
35         // 对每个分配到的 segment 做蝶形
36         for (int seg = start_seg; seg < end_seg; seg++) {
37             int base = seg * h;
38             LL w = 1;
39             for (int k = base; k < base + h/2; k++) {
40                 LL tval = a[k + h/2] * w % p;
41                 a[k + h/2] = (a[k] - tval + p) % p;
42                 a[k] = (a[k] + tval) % p;
43                 w = w * wn % p;
44             }
45         }
46         // 同步到下一层
47         pthread_barrier_wait(barrier);
48     }
49     return nullptr;
50 }
51
52 void NTT_Iteration_Pthread(LL *a, int len, int op, LL g, LL p) {
53     // 位逆序 (单线程)
54     for (int i = 1, j = 0; i < len - 1; i++) {
55         for (int k = len >> 1; (j ^ k) < k; k >>= 1);
56         if (i < j) std::swap(a[i], a[j]);
57     }
58     pthread_barrier_t barrier;

```

```

59 pthread_barrier_init(&barrier, nullptr, MAX_THREADS);
60 // 启动线程
61 pthread_t threads[MAX_THREADS];
62 ThreadArgs args[MAX_THREADS];
63 for (int i = 0; i < MAX_THREADS; i++) {
64     args[i] = { a, len, op, g, p, i, MAX_THREADS, &barrier };
65     pthread_create(&threads[i], nullptr, butterfly_worker, &args[i]);
66 }
67 // 等待结束
68 for (int i = 0; i < MAX_THREADS; i++) {
69     pthread_join(threads[i], nullptr);
70 }
71 pthread_barrier_destroy(&barrier);
72 }
73
74 void NTT_Iteration_multiply_Pthread(const LL *A, const LL *B, LL *C, int n, LL p) {
75     int N = 2*n - 1;
76     int len = 1;
77     while (len < N) len <<= 1;
78     LL inv_len = qpowll(len, p - 2, p);
79
80     // 拷贝并扩展
81     LL *a = new LL[len]();
82     LL *b = new LL[len]();
83     for (int i = 0; i < n; i++) {
84         a[i] = A[i];
85         b[i] = B[i];
86     }
87
88     // 正变换
89     NTT_Iteration_Pthread(a, len, 1, 3, p);
90     NTT_Iteration_Pthread(b, len, 1, 3, p);
91     // 点值乘
92     for (int i = 0; i < len; i++) {
93         a[i] = a[i] * b[i] % p;
94     }
95     // 逆变换
96     NTT_Iteration_Pthread(a, len, -1, 3, p);
97     // 乘上 inv_len
98     for (int i = 0; i < N; i++) {
99         C[i] = a[i] * inv_len % p;
100     }
101
102     delete[] a;
103     delete[] b;
104 }

```

3.1.3 性能分析

<i>time/us</i>		n = 4	n = 131072		
		<i>p</i> = 7340033	<i>p</i> = 7340033	<i>p</i> = 104857601	<i>p</i> = 469762049
算法	<i>NTT_Iteration_multiply</i>	0.00449	83.5684	88.2875	91.4507
	<i>NTT_Iteration_multiply_Pthread</i>	1.88915	29.178	30.7708	31.4566

表 2: 运行结果

性能分析显示, 对于 $n=131072$ 和 $p=7340033$ 的情况, 优化后延迟从 83.57 s 降至 29.18 s, 加速比约 2.86 倍。然而在小规模问题 ($n=4$) 上, 由于线程创建和同步开销, 性能反而下降 (0.00449 s $\rightarrow$ 1.89 s)。值得注意的是, 在尝试超大模数 1337006139375617 时出现了结果错误, 表明朴素方法对大整数运算的支持有限。

3.2 CRT 合并的 Pthread 多线程优化

3.2.1 多模数 CRT 合并 NTT

设计思想 为支持极大模数的 NTT, 本实验采用四个小模数分别对输入执行 NTT 多项式乘法, 点乘后通过中国剩余定理将结果映射回目标大模数域, 最终合并结果满足目标模数 $\text{mod } p$ 。该合并方法适用于同态加密等高精度场景。

代码实现 函数 `NTT_CRT_4mod` 中, 分别在每个模数上调用 `NTT_Iteration_multiply` 处理, 通过 `__int128` 中间计算保证精度, 最后逐项通过 `crt4` 函数进行模数合并, 得到最终结果。

多模数合并 NTT_CRT

```

1  LL inv(LL a, LL p) { return qpowl(a % p, p - 2, p); }
2  LL crt4(LL r1, LL r2, LL r3, LL r4, LL m1, LL m2, LL m3, LL m4, LL mod) {
3      // 如果 mod 恰好等于某个分模, 就直接返回对应余数
4      if (mod == m1) return (r1 % m1 + m1) % m1;
5      if (mod == m2) return (r2 % m2 + m2) % m2;
6      if (mod == m3) return (r3 % m3 + m3) % m3;
7      if (mod == m4) return (r4 % m4 + m4) % m4;
8
9      // 合并前两模  $\rightarrow M_{12}$ 
10     __int128 M1 = m1, M2 = m2;
11     __int128 t12 = (__int128)(r2 - r1) * inv(m1 % m2, m2) % M2;
12     if (t12 < 0) t12 += m2;
13     __int128 x12 = M1 * t12 + r1; // mod M12
14
15     // 合并第三模  $\rightarrow M_{123}$ 
16     __int128 M12 = M1 * M2;
17     __int128 t123 = (__int128)(r3 - x12) * inv((LL)(M12 % m3), m3) % m3;
18     if (t123 < 0) t123 += m3;
19     __int128 x123 = x12 + M12 * t123; // mod M123
20
21     // 合并第四模  $\rightarrow M_{1234}$ 
22     __int128 M123 = M12 * m3;

```

```

23  __int128 t1234 = (__int128)(r4 - x123) * inv((LL)(M123 % m4), m4) % m4;
24  if (t1234 < 0) t1234 += m4;
25  __int128 x1234 = x123 + M123 * t1234; // mod M1234
26
27  // 最后对目标 mod 取一次
28  LL ans = (LL)(x1234 % mod);
29  return (ans + mod) % mod;
30 }
31 void NTT_CRT_4mod(const LL *a, const LL *b, LL *ab, int n, LL LLp) {
32     const int p1 = 998244353;
33     const int p2 = 1004535809;
34     const int p3 = 469762049;
35     const int p4 = 167772161;
36
37     int *a1=new int[n], *b1=new int[n], *r1=new int[2*n];
38     int *a2=new int[n], *b2=new int[n], *r2=new int[2*n];
39     int *a3=new int[n], *b3=new int[n], *r3=new int[2*n];
40     int *a4=new int[n], *b4=new int[n], *r4=new int[2*n];
41
42     for (int i=0;i<n;i++){
43         a1[i]=(a[i]%p1+p1)%p1; b1[i]=(b[i]%p1+p1)%p1;
44         a2[i]=(a[i]%p2+p2)%p2; b2[i]=(b[i]%p2+p2)%p2;
45         a3[i]=(a[i]%p3+p3)%p3; b3[i]=(b[i]%p3+p3)%p3;
46         a4[i]=(a[i]%p4+p4)%p4; b4[i]=(b[i]%p4+p4)%p4;
47     }
48     NTT_Iteration_multiply(a1,b1,r1,n,p1);
49     NTT_Iteration_multiply(a2,b2,r2,n,p2);
50     NTT_Iteration_multiply(a3,b3,r3,n,p3);
51     NTT_Iteration_multiply(a4,b4,r4,n,p4);
52
53     // CRT 合并
54     for (int i=0;i<2*n-1;i++){
55         ab[i] = crt4(
56             r1[i], r2[i], r3[i], r4[i],
57             p1, p2, p3, p4,
58             LLp
59         );
60     }
61
62     // 释放
63     delete[] a1; delete[] b1; delete[] r1;
64     delete[] a2; delete[] b2; delete[] r2;
65     delete[] a3; delete[] b3; delete[] r3;
66     delete[] a4; delete[] b4; delete[] r4;
67 }

```

3.2.2 NTT_CRT_4mod 的 Pthread 多线程优化

设计思想 由于不同模数之间的 NTT 乘法完全独立，天然适合并行。我们创建四个线程，分别计算四个模数上的 NTT 乘法，最后统一在主线程中进行 CRT 合并。

代码实现 在 NTT_CRT_4mod_Pthread 中，通过构造 NTTask 结构体传递参数，每个线程调用 thread_ntt_mul 完成自身模数上的 NTT 乘法，并填充对应输出数组。主线程随后执行 CRT 合并。

多模数合并 NTT 的 Pthread 多线程优化

```

1  struct NTTask { // 单次 NTT 乘法任务参数
2      const LL* a;    // 原始输入 a
3      const LL* b;    // 原始输入 b
4      int n;          // 多项式长度
5      int p;          // 当前模数
6      int g;          // 原根
7      int* res;       // 输出缓冲
8  };
9
10 void* thread_ntt_mul(void* arg) { // 线程执行函数
11     NTTask* task = (NTTask*)arg;
12     int n = task->n, p = task->p, g = task->g;
13     int N = 2 * n - 1, len = 1;
14     while(len < N) len <= 1;
15     int ni = qpow(len, p - 2, p);
16
17     // 分配临时数组
18     int *A = new int[len](), *B = new int[len]();
19     for(int i = 0; i < n; i++){
20         A[i] = (int)((task->a[i] % p + p) % p);
21         B[i] = (int)((task->b[i] % p + p) % p);
22     }
23     // NTT、点乘、逆 NTT
24     NTT_Iteration(A, len, 1, g, p);
25     NTT_Iteration(B, len, 1, g, p);
26     for(int i = 0; i < len; i++) A[i] = (int)((1LL * A[i] * B[i]) % p);
27     NTT_Iteration(A, len, -1, g, p);
28     for(int i = 0; i < N; i++){
29         task->res[i] = (int)((1LL * A[i] * ni) % p);
30     }
31     delete[] A;
32     delete[] B;
33     return nullptr;
34 }
35
36 void NTT_CRT_4mod_Pthread(const LL *a, const LL *b, LL *ab, int n, LL LLp) { // 并行
37     CRT-NTT 乘法
38     static const int p1 = 998244353, p2 = 1004535809,
39     p3 = 469762049, p4 = 167772161;
40     // 为每路分配结果缓冲

```

```

40  int *r1 = new int[2*n], *r2 = new int[2*n],
41  *r3 = new int[2*n], *r4 = new int[2*n];
42
43  // 填充 Task 参数
44  NTTTask tasks[4] = {
45      {a,b,n,p1,g, r1},
46      {a,b,n,p2,g, r2},
47      {a,b,n,p3,g, r3},
48      {a,b,n,p4,g, r4},
49  };
50  pthread_t tids[4];
51  // 创建线程
52  for(int i = 0; i < 4; i++){
53      if(pthread_create(&tids[i], nullptr, thread_ntt_mul, &tasks[i]) != 0){
54          perror("pthread_create");
55          exit(1);
56      }
57  }
58  // 等待线程完成
59  for(int i = 0; i < 4; i++){
60      pthread_join(tids[i], nullptr);
61  }
62
63  // CRT 合并到目标模 LLp
64  for(int i = 0; i < 2*n - 1; i++){
65      ab[i] = crt4(r1[i], r2[i], r3[i], r4[i],
66      p1, p2, p3, p4,
67      LLp);
68  }
69
70  delete[] r1; delete[] r2;
71  delete[] r3; delete[] r4;
72  }

```

3.2.3 性能分析

time/us		n = 4	n = 131072			
		p = 7340033	p = 7340033	104857601	469762049	1337006139375617
算法	NTT_Iteration_multiply	0.00449	83.5684	88.2875	91.4507	结果错误
	NTT_CRT_4mod	0.04388	643.352	644.462	372.949	648.741
	NTT_CRT_4mod_Pthread	0.872592	370.791	373.941	98.1992	374.908

表 3: 运行结果

性能分析表明, 对于 $n=131072$, 多模数方法能正确处理超大模数 (如 1337006139375617), 而朴素方法失败。p=1337006139375617 时的用时从 648.741 s 降至 374.908 s, 加速明显。但相比朴素 Pthread 优化 (27.2072 s), CRT 方法开销更大, 这是其支持大模数必须付出的代价。

4 OpenMP 多线程编程

4.1 朴素 NTT 的 OpenMP 多线程优化

4.1.1 设计思想

OpenMP 实现采用更高级的并行抽象。设计上使用 `#pragma omp parallel for` 自动划分循环迭代，隐式处理线程创建和同步。代码结构更简洁，NTT_Iteration_OpenMP 中通过 `schedule(static)` 保证负载均衡。

4.1.2 代码实现

朴素 NTT 的 OpenMP 多线程优化

```

1  void NTT_Iteration_OpenMP(LL *a, int len, int op, LL g, LL p) {
2      // 1. 位逆序重排
3      for (int i = 1, j = 0; i < len - 1; i++) {
4          for (int k = len >> 1; (j ^ k) < k; k >>= 1);
5          if (i < j) swap(a[i], a[j]);
6      }
7
8      // 2. 分层蝶形运算
9      for (int h = 2; h <= len; h <<= 1) {
10         int segments = len / h;
11         // 根据正/逆变换选择根
12         LL root = (op == 1 ? g : qpowll(g, p - 2, p));
13         LL wn = qpowll(root, (p - 1) / h, p);
14
15         // 并行处理每个 segment
16         #pragma omp parallel for schedule(static)
17         for (int seg = 0; seg < segments; seg++) {
18             int base = seg * h;
19             LL w = 1;
20             for (int k = base; k < base + h/2; k++) {
21                 LL tval = a[k + h/2] * w % p;
22                 a[k + h/2] = (a[k] - tval + p) % p;
23                 a[k] = (a[k] + tval) % p;
24                 w = w * wn % p;
25             }
26         }
27         // 到这里隐式有一个 barrier，同步所有线程
28     }
29 }
30 void NTT_Iteration_multiply_openmp(const LL *A, const LL *B, LL *C, int n, LL p) {
31     int N = 2 * n - 1;
32     int len = 1;
33     while (len < N) len <<= 1;
34     LL inv_len = qpowll(len, p - 2, p);
35 }

```

```

36 // 申请临时数组并初始化
37 vector<LL> a(len), b(len);
38 for (int i = 0; i < n; i++) {
39     a[i] = A[i];
40     b[i] = B[i];
41 }
42 // 其余位置默认 0 (vector 已经自动初始化为 0)
43
44 // 正变换
45 NTT_Iteration_OpenMP(a.data(), len, 1, 3, p);
46 NTT_Iteration_OpenMP(b.data(), len, 1, 3, p);
47
48 // 点值乘, 完全并行
49 #pragma omp parallel for schedule(static)
50 for (int i = 0; i < len; i++) {
51     a[i] = a[i] * b[i] % p;
52 }
53
54 // 逆变换
55 NTT_Iteration_OpenMP(a.data(), len, -1, 3, p);
56
57 // 归一化并写回 C
58 #pragma omp parallel for schedule(static)
59 for (int i = 0; i < N; i++) {
60     C[i] = a[i] * inv_len % p;
61 }
62 }

```

4.1.3 性能分析

time/us		n = 4	n = 131072		
		p = 7340033	p = 7340033	p = 104857601	p = 469762049
算法	NTT_Iteration_multiply	0.00449	83.5684	88.2875	91.4507
	NTT_Iteration_multiply_Pthread	1.88915	29.178	30.7708	31.4566
	NTT_Iteration_multiply_openmp	0.810621	27.3698	27.9615	28.3022

表 4: 运行结果

对比单线程 (83-91 s) 提升约 3 倍, 与 Pthread 版本几乎持平甚至略优, 可能有因 OpenMP 线程池复用减少开销的原因; 小规模数据 $n=4$ 仍存在约 0.8 s 开销, 略逊于串行; 同样在大模数下出现计算错误, 验证了朴素方法的局限性。

4.2 CRT 合并的 OpenMP 多线程优化

4.2.1 设计思想

类似 Pthread 思路, 四个模数上的 NTT 乘法可独立并行, 通过 OpenMP 的 #pragma omp parallel sections 实现。每段通过单独 section 并发执行。CRT 合并过程通过 #pragma omp parallel for 优化。

4.2.2 代码实现

在 NTT_CRT_4mod_openmp 函数中, 利用 OpenMP sections 对四模数 NTT 乘法并发执行, 随后并行遍历合并每个输出系数。

NTT_CRT_4mod_openmp

```

1 // 多模数合并NTT的Openmp并行
2 void NTT_CRT_4mod_openmp(const LL *a, const LL *b, LL *ab, int n, LL LLp) {
3     const LL p1 = 998244353, p2 = 1004535809,
4     p3 = 469762049, p4 = 167772161;
5     const LL g = 3; // 四个模素根均可取 3
6
7     int N = 2*n - 1;
8     int lenR = N; // 合并结果长度
9
10    // 为四路结果分配缓冲
11    vector<LL> r1(lenR), r2(lenR), r3(lenR), r4(lenR);
12
13    // 并行执行四路 NTT 乘法
14    #pragma omp parallel sections
15    {
16        #pragma omp section
17        {
18            NTT_Iteration_multiply_openmp(a, b, r1.data(), n, p1);
19        }
20        #pragma omp section
21        {
22            NTT_Iteration_multiply_openmp(a, b, r2.data(), n, p2);
23        }
24        #pragma omp section
25        {
26            NTT_Iteration_multiply_openmp(a, b, r3.data(), n, p3);
27        }
28        #pragma omp section
29        {
30            NTT_Iteration_multiply_openmp(a, b, r4.data(), n, p4);
31        }
32    }
33
34    // 并行 CRT 合并到目标模 LLp
35    #pragma omp parallel for schedule(static)
36    for (int i = 0; i < lenR; i++) {
37        ab[i] = crt4(
38            r1[i], r2[i], r3[i], r4[i],
39            p1, p2, p3, p4,
40            LLp
41        );
42    }
43 }

```

4.2.3 性能分析

<i>time/us</i>		n = 4	n = 131072			
		<i>p</i> = 7340033	<i>p</i> = 7340033	104857601	469762049	1337006139375617
算法	<i>NTT_Iteration_multiply</i>	0.00449	83.5684	88.2875	91.4507	结果错误
	<i>NTT_CRT_4mod</i>	0.04388	643.352	644.462	372.949	648.741
	<i>NTT_CRT_4mod_Pthread</i>	0.872592	370.791	373.941	98.1992	374.908
	<i>NTT_CRT_4mod_openmp</i>	0.556041	137.028	137.312	103.878	138.969

表 5: 运行结果

未优化的四模数合并 NTT 版本延迟为 643-648 s，OpenMP 优化降至 137-139 s，整体提速约为 4.6 倍。相较于 Pthread 版本（370-374 s），OpenMP 效果更为显著，尤其在大规模数据下。

5 实验总结和思考

通过本次实验，我们发现 NTT 蝶形结构的高度并行性使其非常适合多线程加速。在朴素 NTT 优化中，Pthread 与 OpenMP 均可带来 3 倍左右的性能提升，前者灵活性高但开发复杂，后者开发简洁且性能优秀。在 CRT 多模数合并场景中，OpenMP 的优化效果显著优于 Pthread，尤其在大型数据上表现更为突出，推测原因在于 OpenMP 调度开销更低、分工更合理。综合来看：

1. Pthread 适用于对线程调度与资源控制需求较高的场景；
2. OpenMP 适用于结构明确、并行粒度清晰的数值计算任务；
3. 多模数 NTT 在同态加密等应用中有着重要意义，合理并行优化可显著提升实用性；
4. 线程并行在小规模问题上可能适得其反。

6 链接

Gitee 链接 [Gitee](#)